

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по практической работе №2
по дисциплине «Представление знаний в системах искусственного
интеллекта»
Тема: Построение ЭС с использованием неупорядоченных фактов
(шаблонов) и различных типов условных элементов в антецедентах
правил

Студент гр. 1306

Шаврин А.П

Преподаватель

Сучков А.И.

Санкт-Петербург

2025

Цель работы.

Построить ЭС с использованием неупорядоченных фактов и различных типов условных элементов в антецедентах правил.

Основные теоретические положения.

Неупорядоченные факты представляют собой список взаимосвязанных именованных полей, называемых слотами. Наличие имен полей позволяет осуществлять доступ к полям по именам, в отличие от упорядоченных фактов, где поля специфицируются своим местоположением в факте. Существует два типа слотов: одиночные и мультислоты. Одиночный слот (или просто слот) содержит единственное поле, тогда как мультислот может содержать любое число полей. Для спецификации состава неупорядоченных фактов (содержащихся в них слотов) используются шаблоны, которые задаются конструкцией `deftemplate`. Синтаксис конструкции `deftemplate` определен ниже:

```
(deftemplate <имя шаблона> [“<комментарий>”]
```

```
<Определение слота-1>
```

```
...
```

```
<Определение слота-N>
```

Пример шаблона, содержащего три одиночных слота представлен ниже:

```
(deftemplate object “Шаблон объекта” (slot name) (slot location) (slot weight)).
```

Пример конкретного неупорядоченного факта на основе данного шаблона представлен ниже: (object (name table) (location “room”) (weight 15)). Синтаксис антецедентов правил. Антецедент правила состоит из последовательности условных элементов – УЭ (conditional elements - CEs), которые должны удовлетворяться, чтобы правило было помещено в агенду. В CLIPS используется шесть основных типов условных элементов: УЭ на основе образца, УЭ-проверка, УЭ “И”, УЭ “ИЛИ”, УЭ “НЕ”, УЭ “существует”, УЭ “для всех”, логические УЭ. Ниже рассмотрены наиболее часто используемые типы условных элементов, необходимые для выполнения данной лабораторной работы. УЭ на основе образца (УЭ-образец) – основной и чаще всего используемый тип условного

элемента. Он состоит из совокупности ограничений на поля, масок полей (wildcards) и переменных, используемых в качестве ограничений для фактов и объектов, которые сопоставляются с образцом условного элемента. УЭ-образец удовлетворяется каждой сущностью, которая удовлетворяет его ограничениям. Ограничение на поле представляет собой в общем случае совокупность ограничений, которые используются для проверки единственного поля или слота факта или объекта. Ограничение может состоять из единственного литерала или из нескольких связанных ограничений. Кроме литеральных ограничений, поддерживает три других типа ограничений: связанные ограничения (connective constraints), предикатные ограничения и ограничения возвращаемым значением. Литеральное ограничение задает точное значение (константу) целого, вещественного, символьного или строкового типа, которое должно сопоставляться с полем. При работе с объектами литеральное ограничение задает имя экземпляра. УЭ-образец с литеральными ограничениями не содержит полей масок и переменных. Все ограничения литерального образца должны точно совпадать со всеми полями сопоставляемой сущности. Упорядоченный УЭ-образец, содержащий только литералы, имеет следующий синтаксис: (...). Пример: (data 1 “two”).

УЭ-образец на основе шаблона, содержащий только литеральные ограничения, имеет следующий синтаксис: (() . . . ()). Пример литерального условного элемента для неупорядоченного факта: (person (name Bob) (age 20)). Маски полей (wildcards) используются в условных элементах для указания несущественности некоторого поля или группы полей, что позволяет игнорировать некоторые поля в процессе сопоставления. Считается, что маска сопоставляется с любым значением. Используются одно- и многоместные маски. Одноместная маска обозначаемая символом “?” и сопоставляется с любым значением, занимающим точно одно поле в соответствующем месте сопоставляемой сущности. Многоместная маска обозначаемая парой символов “\$?” и сопоставляется с любыми значениями, занимающими произвольное число полей в сопоставляемой сущности. Одно- и многоместные маски могут

использоваться в одном образце в любых комбинациях. Не допускается использование многоместной маски в одноместном слоте (содержащем единственное поле) неупорядоченных фактов или объектов. Маски могут комбинироваться с литеральными ограничениями в одном УЭ. Пример. Образец (data \$? YELLOW \$?) будет сопоставляться со всеми упорядоченными фактами, содержащими в любом поле, кроме первого, символьное значение YELLOW. В частности, он будет сопоставляться со следующими фактами: (data YELLOW blue red green) (data YELLOW red) (data red YELLOW) (data YELLOW) (data YELLOW data YELLOW). Последний факт будет сопоставляться с образцом дважды, т. к. значение YELLOW содержится в нем два раза. Переменные используются для запоминания значений полей, чтобы их можно было использовать в дальнейшем в других условных элементах антецедента или в операторах консеквента правила. Таким образом, в отличие от масок, переменные позволяют “захватить” значения переменных, содержащихся в сопоставляемой сущности, для дальнейшего использования. Используются одно- и многоместные переменные со следующим синтаксисом: ::= ?, ::= \$?<переменная>, где является символьным типом, но должна начинаться с буквенного символа. В имени переменной не допускается использование кавычек. При первом появлении переменная работает так же, как в маске, т.е. связывается с любым значением в данном поле(ях). Последующие появления переменной требуют, чтобы поле сопоставлялось со связанным значением переменной. Имена переменных являются локальными в пределах каждого правила. Пример. Задано правило: (defrule find-data (data ?x \$?y ?z) => (printout t "?x = " ?x " : " ?y = " ?y " : " ?z = " ?z crlf)) и следующее множество фактов: (data 1 blue) (data 1 blue red) (data 1 blue red 6.9). Тогда при срабатывании правила будет выведен следующий результат: ?x = 1 : ?y = (blue red) : ?z = 6.9 ?x = 1 : ?y = (blue) : ?z = red ?x = 1 : ?y = () : ?z = blue. Ограничения со связками позволяют объединить несколько индивидуальных ограничений и переменных с помощью операций & (и), | (или), и ~ (не). Старшинство операций обычное: ~, &, | - по убыванию. Исключением является случай, когда первым ограничением является

переменная, после которой следует связка &. В этом случае первая переменная трактуется как отдельное ограничение, которое также должно удовлетворяться. Например, ограничение ?x&red|blue трактуется как ?x&(red|blue), а не как (?x&red)|blue. Синтаксис ограничений со связками:

& ... &, | ... |, ~, где - одно- или многоместная переменная, константа или ограничение со связками. Примеры УЭ, содержащих ограничения со связками: (data-A ~blue) (data-A ?x&~green) (data-B (value ~green|red)) (data-B (value ?x&~red&~green)) Предикатное ограничение используется в тех случаях, когда необходимо ограничить поле, основываясь на истинности некоторого булевого выражения. Для этого используется предикатная функция, возвращающая символьное значение FALSE в случае неудачи и другое значение, в случае успеха. Функция вызывается в процессе сопоставления с образцом. Если она возвращает значение FALSE, то ограничение не удовлетворяется, в противном случае – оно удовлетворяется. Предикатное ограничение задается с помощью символа “:”, за которым следует вызов предикатной функции. Предикатное ограничение может использоваться в конъюнкции с ограничением со связками, а также связанной переменной. В последнем случае переменная сначала связывается некоторым значением, а затем к ней применяется предикатное ограничение. В таком варианте предикатные ограничения часто применяются для проверки типов данных. При этом в качестве предикатных функций используются следующие встроенные функции CLIPS: (numberp) - функция возвращает значение TRUE, если имеет тип integer или float, в противном случае возвращается символ FALSE; (floatp) - функция возвращает значение TRUE, если имеет тип float, в противном случае возвращается символ FALSE; (integerp) - функция возвращает значение TRUE, если имеет тип integer, и символ FALSE в противном случае; (symbolp) - функция возвращает значение TRUE, если имеет тип symbol и символ FALSE в противном случае; (stringp) - функция возвращает значение TRUE, если имеет тип string и символ FALSE в противном случае.

Примеры использования предикатных ограничений: (data ?x&:(numberp ?x) (data ?x&~:(symbolp ?x) (data ?x&:(> ?x ?y)).

Ограничение возвращаемым значением использует в качестве ограничения поля значение, возвращаемое некоторой функцией, которая вызывается непосредственно из условного элемента. Возвращаемое значение должно быть одного из примитивных типов. Это значение подставляется непосредственно в образец на позицию, из которой была вызвана функция, как если бы оно было литеральным ограничением. При этом функция вычисляется каждый раз, когда проверяется ограничение. Данное ограничение так же как функция сравнения использует символ “=”. Пусть, например, в базе данных хранятся факты на основе следующего шаблона: (deftemplate data (slot x) (slot y)) Тогда следующий образец будет сопоставляться с фактами, у которых значение второго слота в два раза больше, чем первого.: (data (x ?x) (y =(* 2 ?x))). УЭ-проверка используется для оценки выражений в процессе сопоставления с образцом и имеет следующий синтаксис: (test).

УЭ-проверка удовлетворяется, если вызываемая в нем функция возвращает значение, отличное от FALSE, и не удовлетворяется в противном случае. Использование данного типа УЭ позволяет, в частности, проверять любые соотношения между значениями различных полей (слотов) фактов. Пример. В следующем правиле проверяется, что разность между значениями фактов data и value не меньше трех: (defrule example-1 (data ?x) (value ?y) (test (>= (abs (- ?y ?x)) 3)) =>...).

Постановка задачи.

1. Используя редактор CLIPS (меню File/New, File/Open), сформировать базу данных, содержащую не менее десяти неупорядоченных фактов на основе следующего шаблона:

```
(deftemplate student
  (slot name) ; имя студента
  (slot age) ; возраст
  (slot year) ; год обучения (курс)
  (slot spec) ; специализация
  (slot aver_mark) ; средний балл
)
```

Типы и допустимые значения слотов представлены ниже:

- name (symbol) - любые имена
- age (integer) 17 - 22
- year (integer) 2 - 5
- spec (string) “hard”, “soft”, “ai”
- aver_mark (float) в интервале [3 - 5]

Сохранить конструкции deftemplate и deffacts в файле номер_группы-номер_бригады.clp.

2. Составить в соответствии с вариантом задания, равном номеру бригады, правила, реализующие описанные ниже функции, с использованием заданных типов условных элементов. Правила, соответствующие различным пунктам задания, следует сохранять в разных файлах, добавляя номер пункта в имя файла, чтобы демонстрировать их работу в отчете по отдельности.

2.1. Используя только литеральные ограничения, составить правила для нахождения в БД фактов, удовлетворяющих заданным в таблице 1 условиям, и выдачи соответствующих сообщений.

2.2. Изменить сформированные в п. 2.1. правила путем добавления в антецедент новых условий и изменения выводимых сообщений в соответствии с табл. 1. При реализации новых УЭ использовать УЭ-проверки (test-CE).

2.3. Изменить сформированные в п. 2.2. правила путем добавления в антецеденты предикатных условных элементов для проверки типов значений слотов. Например, для варианта 1 необходимо добавить предикатные УЭ, проверяющие типы значений в слотах <year> и <aver_mark>.

2.4. Изменить сформированные в п. 2.3. правила путем добавления в антецеденты условных элементов с ограничением по возвращаемому значению согласно таблице 1.

Таблица 1. Вариант 3:

Задание	Условие в антецеденте правила	Сообщение, выводимое в консеквенте
2.1	Специализация “ai”	“Студент <name> учится по специализации “ai” на <year> курсе”
2.2	Специализация студента “ai”, возраст – не менее 20 лет	“Возраст студента <name> <age> лет”
2.4	Оканчивает университет в возрасте не младше 25 лет	

Роли:

Алексей Шаврин – выполнение пункта 2.2, придумал окончательное решение для 2.4

Гранкин Захар – выполнение 2.3, 2.4 (придумал первоначальное решение 2.4)

Новицкий Михаил – выполнение пунктов 1 – 2.1.

Выполнение работы.

1. Формирование базы знаний.

Используя редактор CLIPS, была сформирована база данных, содержащая не менее десяти неупорядоченных фактов на основе шаблона данным заданием (рис.1).

```
(deftemplate student
  (slot name)      ; student name (symbol)
  (slot age)       ; student age (integer 17-22)
  (slot year)      ; year of study (integer 2-5)
  (slot spec)      ; specialization (string: "hard", "soft", "ai")
  (slot aver_mark) ; grade point average (float 3.0-5.0)
)

(deffacts students|
  (student (name Alex) (age 21) (year 5) (spec "ai") (aver_mark 5.0))
  (student (name Zakhar) (age 22) (year 2) (spec "ai") (aver_mark 5.0))
  (student (name Michael) (age 20) (year 3) (spec "ai") (aver_mark 5.0))
  (student (name Margarita) (age 19) (year 2) (spec "ai") (aver_mark 4.8))
  (student (name Vlad) (age 17) (year 2) (spec "soft") (aver_mark 4.5))
  (student (name Ilya) (age 19) (year 3) (spec "soft") (aver_mark 4.0))
  (student (name Semen) (age 21) (year 5) (spec "soft") (aver_mark 3.0))
  (student (name Oleg) (age 21) (year 4) (spec "hard") (aver_mark 4.7))
  (student (name Igor) (age 18) (year 2) (spec "hard") (aver_mark 3.5))
  (student (name Alexander) (age 20) (year 4) (spec "hard") (aver_mark 4.0))
)
```

Рисунок 1 – База знаний

Конструкции deftemplate и deffacts в файле 1306-3.clp.

2. Составление правил.

Были составлены правила в соответствии с вариантом задания, реализующие описанные функции с использованием заданных типов условных элементов.

2.1. Используя только литеральные ограничения

Правило находит всех студентов с специализацией “ai”, используя только литеральные ограничения (рис. 2).

```
(defrule find-student-with-ai-specialization
  (student (name ?name) (year ?year) (spec "ai"))
  =>
  (printout t "Студент " ?name " учится по специализации "ai" на " ?year " курсе" crlf)
)
```

Рисунок 2 – Правило для пункта 2.1

2.2. Добавление в антецедент новых условий

Правило из пункта 2.1 было изменено путём добавления в антецедент нового условия (студенты с специализацией “ai” должны быть старше 20 лет) (рис. 3).

```
(defrule find-student-with-ai-specialization-and-over-20-years-of-age
  (student (name ?name) (age ?age) (spec "ai"))
  (test (>= ?age 20))
  =>
  (printout t "Возраст студента " ?name " " ?age " лет" crlf)
)
```

Рисунок 3 – Правило для пункта 2.2

2.3. Добавление в антецеденты предикатных условных элементов для проверки типов значений слотов

Правило из пункта 2.2 было изменено путём добавления в антецеденты предикатного условия проверки условных элементов для проверки типов значений слотов age и spec (рис. 4).

```
(defrule find-student-with-ai-specialization-over-20-years-of-age-and-types-check
  (student
    (name ?name)
    (age ?age & :(integerp ?age))
    (spec ?spec & "ai" & :(stringp ?spec))
  )
  (test (>= ?age 20))
  =>
  (printout t "Возраст студента " ?name " " ?age " лет" crlf)
)
```

Рисунок 4 – Правило для пункта 2.3

2.4. Добавление в антецеденты условных элементов с ограничением по возвращаемому значению.

Правило из пункта 2.3 было изменено путём добавления в антецеденты условных элементов с ограничением по возвращаемому значению. Таким образом получилось правило, которое проверяет, что студент окончит университет в возрасте не младше 25 лет, используя дополнительный факт grad-age. Для этого была модифицирована база знаний (рис. 5).

```

(deftemplate student
  (slot name)      ; student name (symbol)
  (slot age)       ; student age (integer 17-22)
  (slot year)      ; year of study (integer 2-5)
  (slot spec)      ; specialization (string: "hard", "soft", "ai")
  (slot aver_mark) ; grade point average (float 3.0-5.0)
)

(deffacts students
  (student (name Alex) (age 21) (year 5) (spec "ai") (aver_mark 5.0))
  (student (name Zakhar) (age 22) (year 2) (spec "ai") (aver_mark 5.0))
  (student (name Michael) (age 20) (year 3) (spec "ai") (aver_mark 5.0))
  (student (name Margarita) (age 19) (year 2) (spec "ai") (aver_mark 4.8))
  (student (name Vlad) (age 17) (year 2) (spec "soft") (aver_mark 4.5))
  (student (name Ilya) (age 19) (year 3) (spec "soft") (aver_mark 4.0))
  (student (name Semen) (age 21) (year 5) (spec "soft") (aver_mark 3.0))
  (student (name Oleg) (age 21) (year 4) (spec "hard") (aver_mark 4.7))
  (student (name Igor) (age 18) (year 2) (spec "hard") (aver_mark 3.5))
  (student (name Alexander) (age 20) (year 4) (spec "hard") (aver_mark 4.0))
)

(deffacts graduation-ages
  (grad-age 25)
)

(defrule find-student-with-ai-specialization-over-20-years-of-age-with-types-check-and-graduate-from-25-years-old
  (student
    (name ?name)
    (year ?year & :(integerp ?year))
    (age ?age & :(integerp ?age))
    (spec ?spec & "ai" & :(stringp ?spec))
  )
  (test (>= ?age 20))
  (grad-age =(+ ?age (- 5 ?year)))
  =>
  (printout t ?name " оканчивает университет в возрасте не младше 25 лет" crlf)
)

```

Рисунок 5 – Модифицированная база знаний. Правило для пункта 2.4

3. Тестирование разработанной ЭС.

Далее представлены скриншоты тесты программы.

```

f-0      (initial-fact)
f-1      (student (name Alex) (age 21) (year 5) (spec "ai") (aver_mark 5.0))
f-2      (student (name Zakhar) (age 22) (year 2) (spec "ai") (aver_mark 5.0))
f-3      (student (name Michael) (age 20) (year 3) (spec "ai") (aver_mark 5.0))
f-4      (student (name Margarita) (age 19) (year 2) (spec "ai") (aver_mark 4.8))
f-5      (student (name Vlad) (age 17) (year 2) (spec "soft") (aver_mark 4.5))
f-6      (student (name Ilya) (age 19) (year 3) (spec "soft") (aver_mark 4.0))
f-7      (student (name Semen) (age 21) (year 5) (spec "soft") (aver_mark 3.0))
f-8      (student (name Oleg) (age 21) (year 4) (spec "hard") (aver_mark 4.7))
f-9      (student (name Igor) (age 18) (year 2) (spec "hard") (aver_mark 3.5))
f-10     (student (name Alexander) (age 20) (year 4) (spec "hard") (aver_mark 4.0))

```

Рисунок 6 – Основные факты.

```

CLIPS> (run)
FIRE    1 find-student-with-ai-specialization: f-4
Студент Margarita учится по специализации "ai" на 2 курсе
FIRE    2 find-student-with-ai-specialization: f-3
Студент Michael учится по специализации "ai" на 3 курсе
FIRE    3 find-student-with-ai-specialization: f-2
Студент Zakhar учится по специализации "ai" на 2 курсе
FIRE    4 find-student-with-ai-specialization: f-1
Студент Alex учится по специализации "ai" на 5 курсе
CLIPS>

      0      find-student-with-ai-specialization: f-4
      0      find-student-with-ai-specialization: f-3
      0      find-student-with-ai-specialization: f-2
      0      find-student-with-ai-specialization: f-1

```

Рисунок 7 – Вывод теста правила 2.1. Агенда для правила 2.1.

```

CLIPS> (run)
FIRE    1 find-student-with-ai-specialization-and-over-20-years-of-age: f-3
Возраст студента Michael 20 лет
FIRE    2 find-student-with-ai-specialization-and-over-20-years-of-age: f-2
Возраст студента Zakhar 22 лет
FIRE    3 find-student-with-ai-specialization-and-over-20-years-of-age: f-1
Возраст студента Alex 21 лет
CLIPS>

      0      find-student-with-ai-specialization-and-over-20-years-of-age: f-3
      0      find-student-with-ai-specialization-and-over-20-years-of-age: f-2
      0      find-student-with-ai-specialization-and-over-20-years-of-age: f-1

```

Рисунок 8 – Вывод теста правила 2.2. Агенда правила 2.2.

```

CLIPS> (run)
FIRE    1 find-student-with-ai-specialization-over-20-years-of-age-and-types-check: f-3
Возраст студента Michael 20 лет
FIRE    2 find-student-with-ai-specialization-over-20-years-of-age-and-types-check: f-2
Возраст студента Zakhar 22 лет
FIRE    3 find-student-with-ai-specialization-over-20-years-of-age-and-types-check: f-1
Возраст студента Alex 21 лет
CLIPS>

      0      find-student-with-ai-specialization-over-20-years-of-age-and-types-check: f-3
      0      find-student-with-ai-specialization-over-20-years-of-age-and-types-check: f-2
      0      find-student-with-ai-specialization-over-20-years-of-age-and-types-check: f-1

```

Рисунок 9 – Вывод теста правила 2.3. Агенда правила 2.3.

```

f-0      (initial-fact)
f-1      (student (name Alex) (age 21) (year 5) (spec "ai") (aver_mark 5.0))
f-2      (student (name Zakhar) (age 22) (year 2) (spec "ai") (aver_mark 5.0))
f-3      (student (name Michael) (age 20) (year 3) (spec "ai") (aver_mark 5.0))
f-4      (student (name Margarita) (age 19) (year 2) (spec "ai") (aver_mark 4.8))
f-5      (student (name Vlad) (age 17) (year 2) (spec "soft") (aver_mark 4.5))
f-6      (student (name Ilya) (age 19) (year 3) (spec "soft") (aver_mark 4.0))
f-7      (student (name Semen) (age 21) (year 5) (spec "soft") (aver_mark 3.0))
f-8      (student (name Oleg) (age 21) (year 4) (spec "hard") (aver_mark 4.7))
f-9      (student (name Igor) (age 18) (year 2) (spec "hard") (aver_mark 3.5))
f-10     (student (name Alexander) (age 20) (year 4) (spec "hard") (aver_mark 4.0))
f-11     (grad-age 25)

CLIPS> (run)
FIRE    1 find-student-with-ai-specialization-over-20-years-of-age-with-types-check-and-graduate-from-25-years-old: f-2,f-11
Zakhar оканчивает университет в возрасте не младше 25 лет
CLIPS>

```


Рисунок 10 – Все основные факты с дополнительным фактом для правила 2.4.
Вывод теста правила 2.4. Агенда правила 2.4.

Выводы.

В ходе выполнения лабораторной работы изучены принципы работы с неупорядоченными фактами и различными типами условных элементов в системе CLIPS. Была создана экспертная система для анализа данных о студентах, которая на основе структурированной информации выдает аналитические выводы о возрасте выпуска и специализации студентов. Система динамически проверяет корректность входных данных и применяет сложные логические условия для фильтрации результатов.

В ходе работы столкнулись с проблемой интерпретации требования пункта 2.4, где необходимо было реализовать проверку на строгое равенство, в то время как логика задачи требовала проверки нестрогого неравенства (возраст выпуска не младше 25 лет). Для решения этой проблемы был разработан подход с использованием дополнительного факта grad-age.

Идея решения заключалась в следующем: создается набор фактов graduation-ages, определяющих допустимые возраста выпуска (≥ 25 лет), после чего проверяется, попадает ли вычисленный возраст окончания университета в это множество. Учитывая ограничения на курсы (2-5) и возраст (17-22), расчет показывает, что максимальный возможный возраст выпуска составляет 25 лет ($22 + 3$ года обучения при поступлении на 2 курс). Таким образом, в правиле применяется точное сравнение вычисленного возраста выпуска с фактом grad-age, что эквивалентно проверке на принадлежность к допустимому множеству значений.

Данный подход позволил корректно реализовать требование пункта 2.4, используя формально строгое равенство для проверки, по существу, нестрогого условия.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД

Файл 1306-3.clp.

```
(deftemplate student
  (slot name)      ; student name (symbol)
  (slot age)       ; student age (integer 17-22)
  (slot year)      ; year of study (integer 2-5)
  (slot spec)      ; specialization (string: "hard", "soft", "ai")
  (slot aver_mark) ; grade point average (float 3.0-5.0)
)

(deffacts students
  (student (name Alex) (age 21) (year 5) (spec "ai") (aver_mark
5.0))
  (student (name Zakhar) (age 22) (year 2) (spec "ai") (aver_mark
5.0))
  (student (name Michael) (age 20) (year 3) (spec "ai") (aver_mark
5.0))
  (student (name Margarita) (age 19) (year 2) (spec "ai")
(aver_mark 4.8))
  (student (name Vlad) (age 17) (year 2) (spec "soft") (aver_mark
4.5))
  (student (name Ilya) (age 19) (year 3) (spec "soft") (aver_mark
4.0))
  (student (name Semen) (age 21) (year 5) (spec "soft") (aver_mark
3.0))
  (student (name Oleg) (age 21) (year 4) (spec "hard") (aver_mark
4.7))
  (student (name Igor) (age 18) (year 2) (spec "hard") (aver_mark
3.5))
  (student (name Alexander) (age 20) (year 4) (spec "hard")
(aver_mark 4.0))
)

(deffacts graduation-ages
  (grad-age 25)
)
```

Файл 1306-3-2.1.clp.

```
(defrule find-student-with-ai-specialization
  (student (name ?name) (year ?year) (spec "ai"))
  =>
  (printout t "Студент " ?name " учится по специализации "ai" на "
?year " курсе" crlf)
)
```

Файл 1306-3-2.2.clp.

```
(defrule find-student-with-ai-specialization-and-over-20-years-of-
age
  (student (name ?name) (age ?age) (spec "ai"))
  (test (>= ?age 20))
  =>
  (printout t "Возраст студента " ?name " " ?age " лет" crlf)
)
```

Файл 1306-3-2.3.clp.

```
(defrule find-student-with-ai-specialization-over-20-years-of-age-
and-types-check
  (student
    (name ?name)
    (age ?age & : (integerp ?age))
    (spec ?spec & "ai" & : (stringp ?spec))
  )
  (test (>= ?age 20))
=>
  (printout t "Возраст студента " ?name " " ?age " лет" crlf)
)
```

Файл 1306-3-2.4.clp.

```
(defrule find-student-with-ai-specialization-over-20-years-of-age-
with-types-check-and-graduate-from-25-years-old
  (student
    (name ?name)
    (year ?year & : (integerp ?year))
    (age ?age & : (integerp ?age))
    (spec ?spec & "ai" & : (stringp ?spec))
  )
  (test (>= ?age 20))
  (grad-age = (+ ?age (- 5 ?year)))
=>
  (printout t ?name " оканчивает университет в возрасте не младше
25 лет" crlf)
)
```